

Does compression change behavior?

When an agent's memory gets compacted,
does it still **do** the same things?

Agents forget.

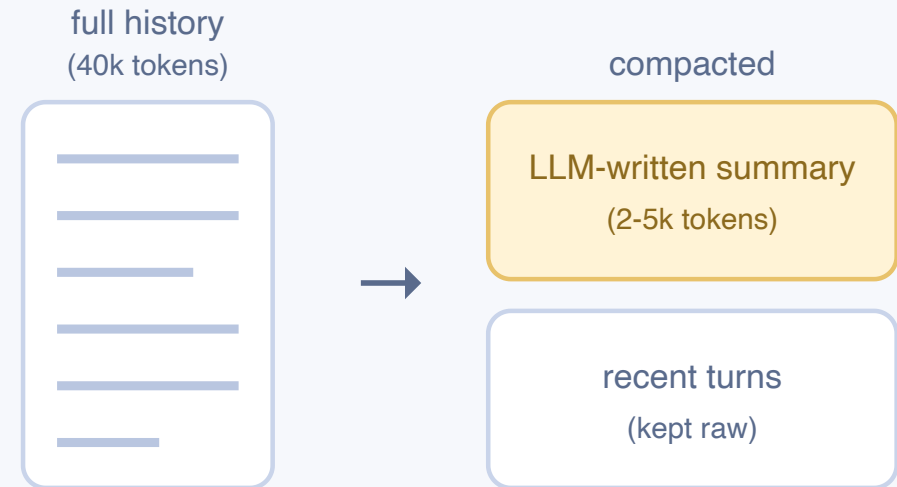
Long tasks overflow the context window.

Every system does the same thing:

summarize, keep going.

And checks the same thing:

did the facts survive?



We ask a different question.

**Would the agent still take
the same next action?**

A perfect compaction is invisible:
the agent acts as if nothing was removed.

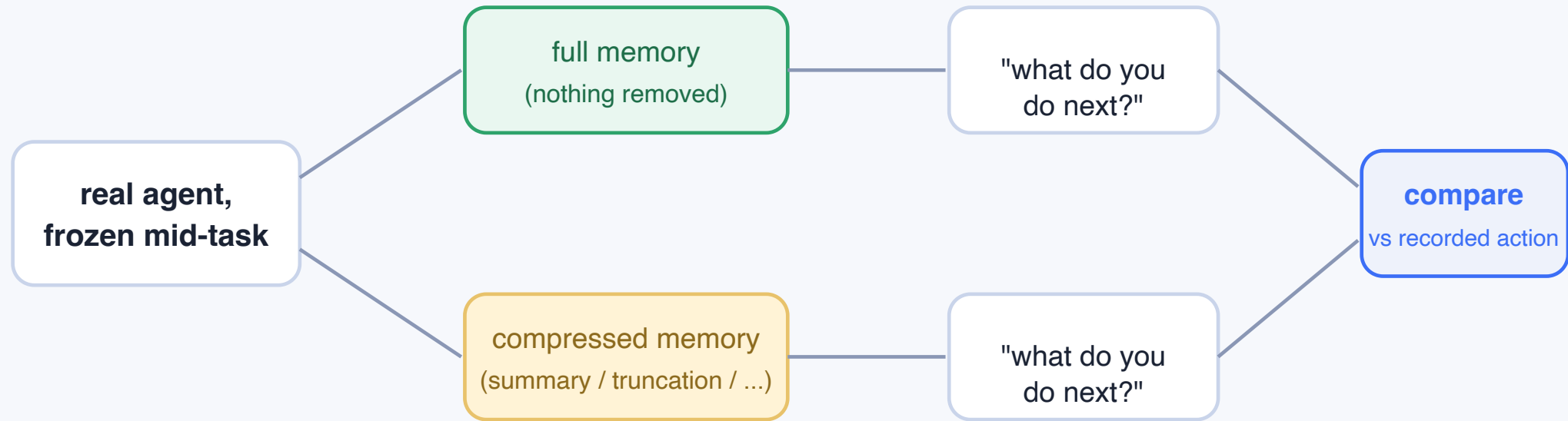
✓ **what everyone checks**

"are the facts still there?"

? **what we check**

"same next action?"

The test



The original run already recorded the true answer. We count matches.

64 stopping points taken from our own agents' runs; at each one we sample the next action 8 times per memory version. Sampling is random, so even full memory does not always repeat itself. That natural wobble is our baseline, and we only trust differences bigger than it.

Summaries fail this test.

How often the agent repeats its real next action:

full memory (100% of tokens): 65%

just keep recent tokens (25% of tokens): 66%

LLM-written summary: 46%

The default in every harness is the worst option we measured.

65% is the ceiling: sampling is random, so even full memory does not always repeat itself. Keeping just the recent quarter of the tokens matches that ceiling; the summary falls 19 points below it.

Copy exactly. Never paraphrase.

Same information, same size budget:

```
grep -rn "auth" src/utils/  
cat src/utils/auth.py
```

byte-exact copies

agreement 0.71

```
GREP auth src/  
READ auth.py
```

same info, rewritten

agreement 0.55

Holds at every budget we tried (keep 25%, 50%, or 75% of the history).



Spend an
entire LLM call
writing a careful,
fluent
summary of the history



Delete
old
tokens. Do
nothing else.

46% vs 66% agreement.

A tiny core carries the behavior.

How small can the memory get? We deleted **everything** except one thing: the bare list of commands the agent itself had run, trimmed to 2% of the original tokens.

full memory (100% of tokens): 65%

only its own past commands (2% of tokens): 57%

LLM summary: 46%

A bare list of its own actions preserves most of the behavior.

A fluent summary, using far more space, preserves less.

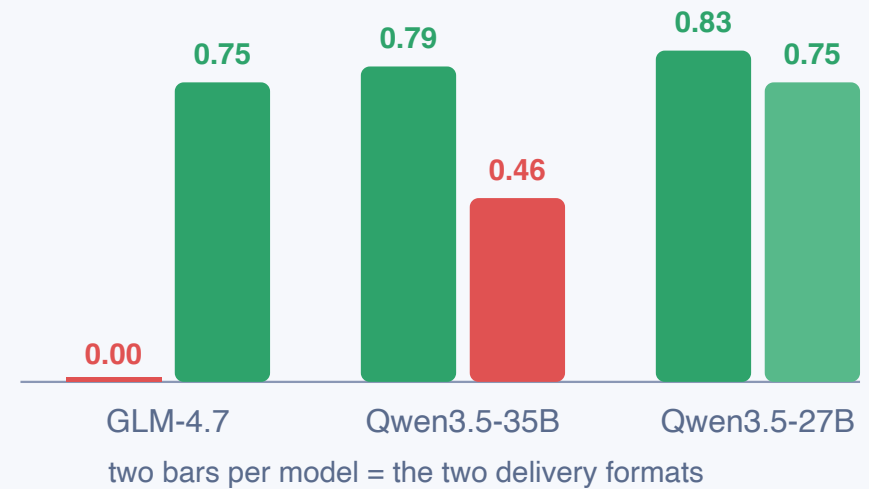
Format cliffs

The **same summary text**, delivered two ways.

One model is fine. Another outputs gibberish.

**There is no universal
compaction format.**

fraction of coherent replies, same content



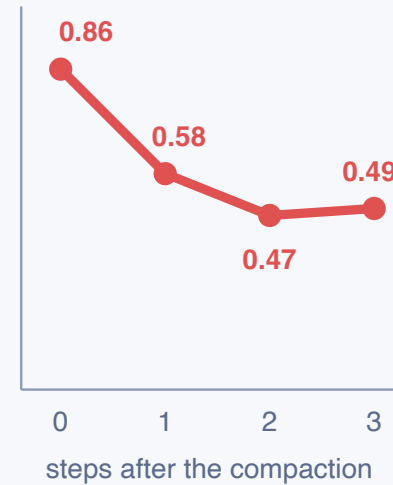
The training mismatch

The first compression does nearly all the damage.

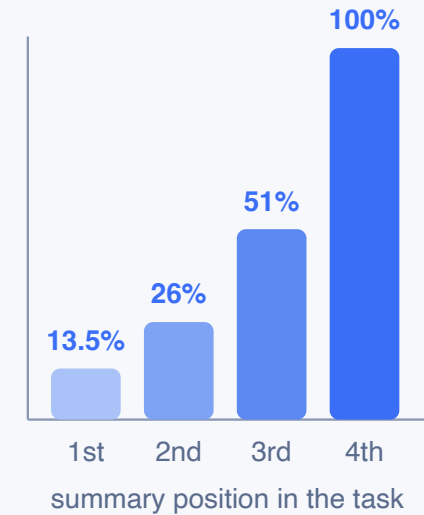
But RL schemes give that first summary the **least** learning signal.

The one that hurts most learns least.

damage after one compaction



training signal per summary

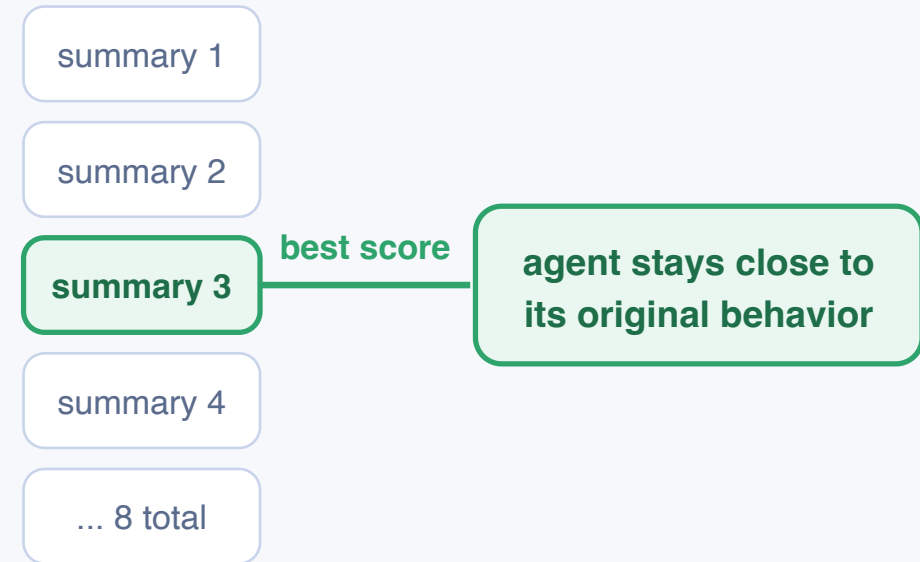


Using the score to pick better summaries

Write **8 candidate summaries** of the same history.

Score each one. Keep the best.

That one pick recovers about half of what compression lost.



What's next

01 Tie compression to task success.

Run live episodes with each compaction policy and compare completion rates.

02 Continue scaling.

Stronger models, longer contexts, non-coding tasks: see how far the rules hold.

03 Train a compressor with the behavior score.

Selection already works; the first small training run showed no effect, a larger one is queued.

**Keep the behavior,
not just the facts.**

github.com/ayushnangia/does-compression-change-behavior